

Mini Cajero · Parte 2 — Ordena tu cajero con funciones

JavaScript · Node.js · Taller práctico · Continuación del taller «Construye una Calculadora — Mini Cajero»

En la Parte 1 construiste un cajero que atiende operaciones una tras otra. Funciona, pero todo vive junto dentro del bucle: pedir datos, decidir la operación, calcular y mostrar.

En esta Parte 2 vas a usar lo que aprendiste en la clase de funciones para **ordenar ese mismo cajero**: cada tarea en su propia función, con nombre, lista para reutilizarse. El código lo sigues escribiendo tú: la guía te acompaña con instrucciones y pistas.

Lo que vas a construir

El mismo cajero de la Parte 1, organizado en funciones: una **pide** números, otra **calcula**, otra **muestra** y una **coordina** a las demás.

Al final, tu bucle quedará de muy pocas líneas y, cuando el negocio cambie una regla, sabrás exactamente qué función tocar.

Antes de empezar: piensa en esto

Nequi no hace sus cálculos en un solo lugar: los usa cuando mandas un pago, en el extracto de fin de mes y en el chat de soporte cuando alguien pregunta por un movimiento.

Si ese cálculo estuviera copiado en los tres lugares y mañana cambiara una regla, ¿qué pasaría? Ahora mira tu cajero: tu cálculo está pegado dentro del bucle. ¿Podrías usarlo en otro lugar sin copiarlo y pegarlo?

Paso 0 · Prepara tu archivo

Vas a trabajar en un archivo nuevo para que tu cajero de la Parte 1 quede intacto.

1. Abre en VS Code la carpeta del proyecto de la Parte 1: la que tiene `cajero.js`, `package.json` y la carpeta `node_modules`.
2. En esa misma carpeta, crea un archivo nuevo llamado `cajero-v2.js`.
3. Copia todo el contenido de `cajero.js` y pégalo en `cajero-v2.js`.
4. En la terminal, ejecuta `node cajero-v2.js` y comprueba que funciona igual que antes.

Pista

No necesitas instalar `prompt-sync` otra vez: ya está en tu carpeta. Desde ahora, todo el trabajo va en `cajero-v2.js` y lo ejecutas con `node cajero-v2.js`.

El orden de un archivo con funciones

Igual que en la clase de la tienda: **primero las herramientas, después el uso.**

1. Arriba: la línea de `prompt-sync` (sigue siendo la primera del archivo).
2. En el medio: tus funciones.
3. Abajo: la variable `activo`, el bucle y el mensaje de despedida.

Sin IA aquí

Desde este paso hasta la Parte 4, el código lo escribes tú: sin chat de IA y sin sugerencias en línea de Copilot. La IA entra al final, en un momento específico, como revisora.

Parte 1 · Pedir números sin repetirte

Para pensar

Mira cómo pides `numero1` y `numero2` en tu cajero. ¿Qué se repite? Si mañana necesitas pedir un tercer número, ¿qué tendrías que copiar otra vez?

Crea la función `pedirNumero`

1. Debajo de la línea de `prompt-sync`, define una función llamada `pedirNumero` que reciba un parámetro llamado `mensaje`.
2. Adentro, usa `prompt()` con ese mensaje, convierte lo que llegue con `Number()` y **devuelve** el número con `return`.
3. En tu cajero, cambia la forma de pedir `numero1` y `numero2`: ahora cada variable guarda lo que devuelve `pedirNumero`, y a cada llamada le envías un mensaje distinto como argumento.

Pista

`prompt()` ya recibía un mensaje entre paréntesis. Tu función hace lo mismo: el mensaje entra como **argumento**, llega al **parámetro** `mensaje` y de ahí tú se lo pasas a `prompt()`.

Prueba esto y observa

Borra por un momento la palabra `return` de `pedirNumero` y ejecuta con 5, + y 3. El cajero te pide los datos normalmente... pero el resultado ya no es 8. ¿Qué sale?

Cuando termines de observar, vuelve a poner el `return`.

Para pensar

Si `prompt()` sí recibió tu 5, ¿por qué el cajero no lo pudo usar? ¿Qué diferencia hay entre que la función **tenga** el número y que lo **entregue**?

Parte 2 · Separar el cálculo

Para pensar

Tu estructura condicional (la que decide si suma, resta, multiplica o divide) está mezclada con todo lo demás. ¿Qué datos necesita para trabajar? ¿Y qué debería entregar cuando termine?

Crea la función `calcular`

1. Define una función llamada `calcular` que reciba tres parámetros, en este orden: `numero1`, `operacion` y `numero2`.
2. Mueve adentro de la función tu estructura condicional de la Parte 1.
3. En cada camino (+, -, *, /), en lugar de guardar el resultado en una variable, **devuélvelo** con `return`.
4. En el caso final, el de los símbolos inválidos, devuelve el mensaje "`Operación no válida`".
5. En tu cajero, guarda en la variable `resultado` lo que devuelva `calcular`, enviándole los tres datos.

Pista

Es la misma idea de `calcularEnvio` en la clase: **cada camino** del condicional tiene su propio `return`. Cuando se ejecuta un `return`, la función termina ahí mismo.

Para pensar

¿Por qué `calcular` no debería tener ningún `console.log()`? Piensa en Nequi: el resultado de un cálculo, ¿lo usa una persona que lo lee o el código que lo necesita para seguir trabajando?

Prueba esto y observa

Donde llamas a `calcular`, cambia por un momento el orden de los argumentos: pon `numero2` en el lugar de `operacion`. Ejecuta con 5, + y 3.

No sale ningún error rojo... pero ¿qué responde el cajero? ¿Por qué? Después, deja el orden correcto.

Parte 3 · Mostrar y coordinar

🤔 Para pensar

Ya tienes una función que pide y otra que calcula. Falta alguien que **muestre** el resultado y alguien que ponga a todas a trabajar, en orden, cada vez que llega una operación.

Crea la función `mostrarResultado`

1. Define una función llamada `mostrarResultado` que reciba un parámetro: `resultado`.
2. Adentro, muéstralo con `console.log()` y una plantilla de texto. Por ejemplo, que se vea así: `Resultado: 8`.

Crea la función `atenderOperacion`

1. Define una función llamada `atenderOperacion`, sin parámetros.
2. Adentro, en este orden: pide `numero1` con `pedirNumero`, pide la `operacion` con `prompt()`, pide `numero2` con `pedirNumero`, calcula con `calcular` (guardando lo que devuelve) y muestra con `mostrarResultado`.
3. Dentro de tu bucle, borra todo el código que ya pasó a `atenderOperacion` y deja en su lugar una sola llamada: `atenderOperacion()`.
4. Ejecuta y comprueba que tu cajero hace lo mismo que antes.

⚠️ Cuidado: `activo` se queda afuera

La pregunta de si quiere hacer otra operación y el cambio de `activo` a `false` **se quedan en el bucle**, no dentro de `atenderOperacion`.

`atenderOperacion` atiende **una** operación. Decidir si el cajero sigue abierto es otra tarea. Además, si declaras `activo` dentro de una función, se vuelve una variable local y el bucle de afuera no la puede ver.

👁️ Si ves `Resultado: undefined`

Revisa `calcular`: seguramente quedó un `console.log()` en lugar de un `return`. Es la diferencia entre **mostrar** y **entregar** que viste en clase.

🤔 Para pensar

`mostrarResultado` no tiene `return`. ¿Está mal? Y mira tu bucle ahora: ¿cuántas líneas tiene? Si alguien nuevo abre tu archivo, ¿puede entender qué hace el cajero solo leyendo los nombres de las funciones?

Parte 4 · Llega un cambio del negocio

Gerencia revisó el cajero: cuando alguien divide entre 0, el programa muestra `Infinity`. Un cliente no entiende eso.

Nueva regla: si se intenta dividir entre 0, el resultado debe ser el mensaje "No se puede dividir entre 0".

Para pensar

Antes de tocar nada: ¿cuál de tus funciones tiene que cambiar? ¿`pedirNumero`, `calcular`, `mostrarResultado` o `atenderOperacion`?

Aplica el cambio

1. Solo en la función que elegiste, dentro del camino de la división, agrega una condición: si `numero2` es 0, devuelve el mensaje; si no, devuelve la división.
2. Prueba `10 / 0` y también `10 / 2`, para confirmar que la división normal sigue funcionando.

Pista

Si en la Parte 1 ya habías resuelto este reto dentro del bucle, no lo borres sin más: **muévelo** a la función donde corresponde.

Para pensar

¿Cuántas funciones cambiaste? ¿Las demás se enteraron del cambio?

Esto tiene nombre: responsabilidad única

Cada función tiene **una sola tarea** y, por eso, **una sola razón para cambiar**. Cuando llega una regla nueva, cambia la función que sabe de esa regla y las demás ni se enteran.

Momento IA · La IA revisa, no escribe

Tu cajero ya funciona y está organizado. Ahora vas a pedir una **revisión**, como se hace en un equipo real de desarrollo: tú escribiste el código, la IA opina y tú decides.

1 · Pide la revisión

Usa Copilot Chat en VS Code o el chat de IA en el navegador. Copia este mensaje y pega tu código donde se indica:

Soy principiante en JavaScript. Hice un cajero en Node.js con prompt-sync y lo organicé en funciones: `pedirNumero`, `calcular`, `mostrarResultado` y `atenderOperacion`. [pega aquí tu código completo]. No lo reescribas. Revísalo y dime: 1) si hay casos donde falla, 2) si cada función hace una sola tarea, 3) si los nombres de las funciones, parámetros y variables dicen lo que hacen. Usa solo lo que he visto: variables, if/else, while, for, arreglos, objetos y funciones con return.

2 · Valida lo que te responde

- Si la IA dice que algo falla, **pruébalo en tu cajero**. Si al ejecutarlo no falla, la IA se equivocó.
- Si te propone código que no conoces (por ejemplo `=>`, `?`, `switch` o `try`), no lo copies: pregúntale por qué y pídele que lo explique con lo que ya sabes.
- Lo que **no** le entregas a la IA: escribir tu código y decidir si su crítica es correcta.

Para pensar

La IA te dice que tu cajero está perfecto. ¿Ya está perfecto? ¿Quién tiene la última palabra?

✓ Lo que acabas de lograr

Tu cajero hace lo mismo que en la Parte 1, pero ahora cada tarea tiene dueño:

Función	Recibe	Entrega (return)	Su única tarea
<code>pedirNumero</code>	un mensaje	un número	Pedir un número
<code>calcular</code>	dos números y una operación	el resultado o un mensaje	Calcular
<code>mostrarResultado</code>	el resultado	nada	Mostrar en pantalla
<code>atenderOperacion</code>	nada	nada	Coordinar a las demás

✓ En resumen

- Las funciones que **calculan** entregan con `return`; solo una función **muestra**.
- Una función **coordina** a las demás, y tu bucle quedó corto y fácil de leer.
- Un cambio del negocio se hizo en **un solo lugar**.

Desde ahora, frente a cualquier problema de negocio, la primera pregunta es: **¿qué tareas hay y qué función se encarga de cada una?**